

OTTER BANK ・ 2026-09-28 時点の main

構成マップ

Next.js 16 と Rails 8.1 API で動く家計簿アプリの全体像、認証の流れ、データと実績の仕組みを 12 枚で見返す

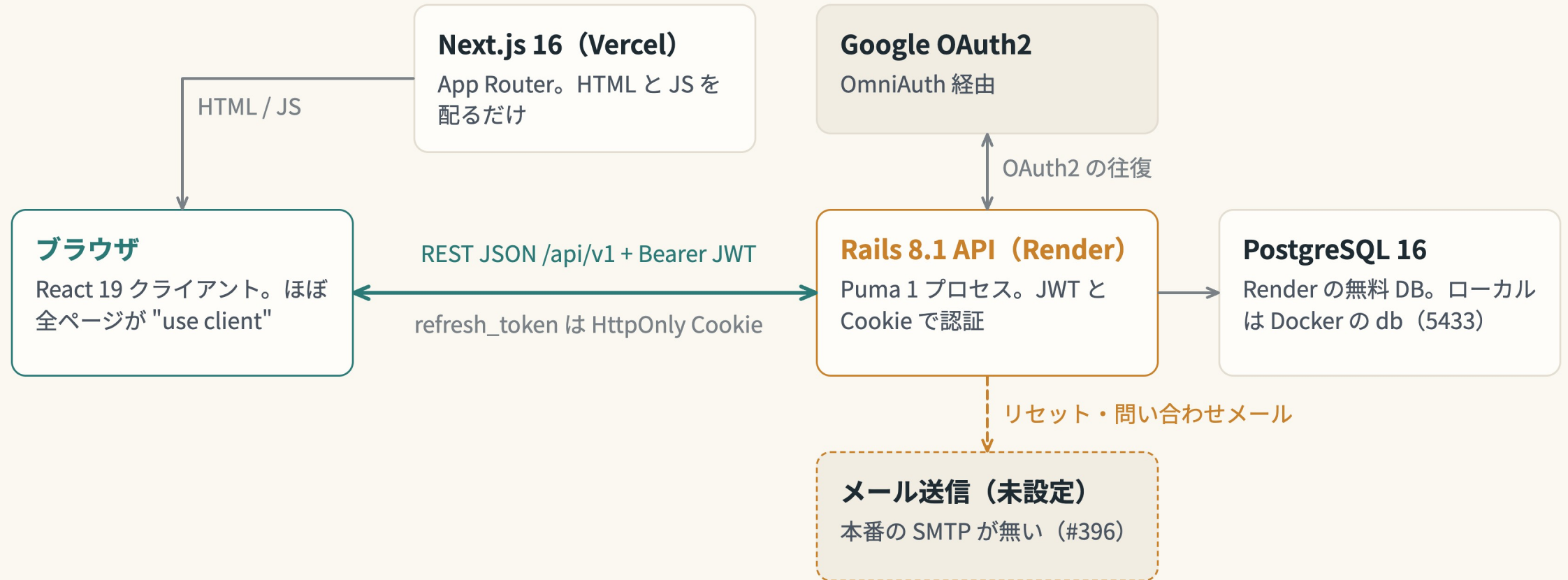
全体像

認証フロー

データモデルと実績

自分用の全体把握 ・ github.com/jinxin4869/Otter_Bank

全体像: フロントは Vercel、API と DB は Render



1 リクエストの通り道: 画面から DB まで 7 層

フロント (front/src)

1. ページ

`app/<route>/page.tsx`

useAuth で token を取り、api.xxx() を呼ぶ



2. エンドポイント定義

`lib/api.ts`

auth / transactions / achievements / posts / contacts をオブジェクトで束ねる



3. fetch ラッパー

`lib/api-client.ts`

Bearer 付与、JSON 化、失敗は `ApiError(message, code)` に整形

バック (back/app)

4. ルーティング

`config/routes.rb`

全部 /api/v1 配下、46 アクション



5. コントローラー

`controllers/api/v1/*`

ApplicationController が JWT を検証。concerns: ExceptionHandler / PostLookup / AchievementJson



6. モデル・サービス

`models/*` ・ `services/*`

AchievementService (実績判定)、JsonWebToken (署名)



7. PostgreSQL

14 テーブル。schema.rb が正

ディレクトリ: 迷ったらここを見る

front/src

パス	役割
app/<route>/	ページ。専用部品は _components/ に置く
app/board/_components/	投稿カード・作成/編集モダル・コメント
components/	Header, OtterAnimation, グラフ, チュートリアル
components/ui/	shadcn/ui (生成物。手で直さない)
hooks/	useAuth, useAchievements
lib/	api.ts, api-client.ts, schemas (zod) , tier.tsx
types/	API 型 (snake) と内部型 (camel) と変換関数

back

パス	役割
app/controllers/api/v1/	15 コントローラー。全部 Api::V1
app/controllers/concerns/	例外整形、投稿の読み込み、実績 JSON
app/models/	14 モデル。User が一番大きい (232 行)
app/services/	AchievementService (399 行) , JsonWebToken
app/mailers/	パスワードリセット・問い合わせ確認
config/initializers/	cors, rack_attack, omniauth, jwt_secret
spec/	RSpec 37 ファイル (request / model / service)

ルート直下: compose.yml (front 4000 / back 3000 / db 5433)、render.yaml (Render の API と DB)、docs/ (PRD・ARCHITECTURE・DESIGN-SYSTEM・ER 図)。back/test/ は使われていない minitest の残骸 (#406)

画面と API の対応: 9 画面、うち 3 つが本体

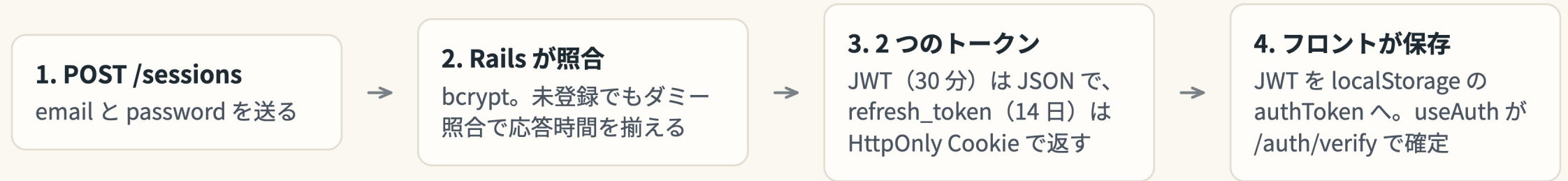
ダッシュボード	/dashboard	GET/POST/DELETE transactions, GET achievements	取引登録の応答に newly_unlocked_achievements
掲示板	/board	posts, comments, like/unlike, bookmark, increment_views	20 件ずつ「もっと見る」
実績	/collection	GET achievements	summary に growth_stage
ログイン	/login	POST sessions, GET auth/google	oauth_error クエリで失敗表示
新規登録	/register	POST users	登録後にチュートリアル
Google 戻り先	/auth/callback	POST auth/refresh	Cookie から JWT を受け取る
パスワード再設定	/reset-password, /[token]	POST auth/reset-password, /confirm	メールのリンクで token を受ける
お問い合わせ	/contact	POST contacts	ログイン不要
トップ・規約・使い方	/, /terms, /privacy, /tutorial	なし	静的

API だけあって画面が無いもの

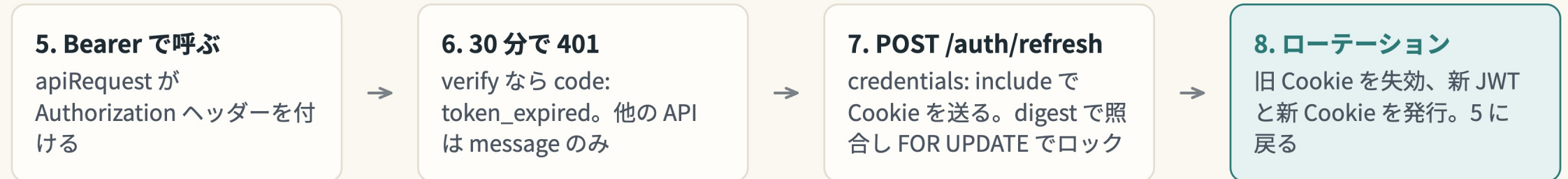
/budgets（予算）・/savings_goals（貯金目標）・/guest_sessions・PATCH /transactions/:id・PATCH /achievements/:id・PATCH/DELETE /user。予算と貯金目標が無いので実績 5 件が解除不能（#398）

認証 1: JWT は 30 分、リフレッシュ Cookie は 14 日

ログイン



以降のリクエストと期限切れ



ログアウトはDELETE /sessionsでCookieのrefresh_tokenを失効させ、localStorageを消す。署名鍵はJWT_SECRET (Renderが生成)。Cookieのdigestも同じ鍵でHMAC

認証 2: Google ログインは URL に JWT を載せない

1. ボタンを押す

window.location を GET
/api/v1/auth/google に向ける（画面
遷移）



2. OmniAuth へ

Rails が /auth/google_oauth2 にリダイ
レクト。scope は email と profile



3. Google で認証

同意画面。キャンセルは on_failure で
/login?oauth_error=cancelled へ

4. コールバック

GET /auth/google/callback。
User.find_or_create_from_oauth:
provider+uid → email の順で既存を探
し、無ければ作成



5. Cookie だけ持って戻る

refresh_token を HttpOnly Cookie にセ
ットし、302 で
FRONTEND_URL/auth/callback へ。
JWT は渡さない



6. フロントで引き換え

/auth/callback が POST /auth/refresh
を 1 回だけ呼び、JWT を受け取って
login()。2 秒後に /dashboard

5 はトップレベルの画面遷移なので Cookie が確実に保存される。これがメール/パスワード経路（fetch）との違いで、Google ログインだけ 30 分後の更新が成功する

認証で見つかった穴は3つ。どれも issue 化済み

#395 ・ 優先度:高

Cookie が保存されない

api.auth.login と register に credentials: 'include' が無い。別オリジンの fetch なので Set-Cookie が捨てられ、30 分後の更新が必ず失敗する

直し方: 2箇所 credentials を足すだけ

#397 ・ 優先度:高

操作中の 401 を拾わない

リフレッシュはマウント時だけ。画面を開いたまま 30 分経つと、取引登録も投稿も失敗し続け、リロードするしかない

直し方: apiRequest で token_expired を受けたら refresh して 1 回だけ再試行。サーバーの 401 にも code を付ける

#396 ・ 優先度:高

リセットメールが出ない

本番に SMTP 設定が無い。画面は「送信しました」と出るが届かないので、パスワードを忘れたら戻れない

直し方: production.rb に smtp_settings、render.yaml に環境変数

残りの監査結果は #398～#410（優先度:高 1 件、中 4 件、低 8 件）。タイムゾーン UTC（#399）は連続記録の判定に効く

データモデル: 14 テーブル、すべて users にぶら下がる

user email (unique) , username, name, password_digest (OAuth のみなら空) , last_sign_in_at / current_sign_in_at, reset_password_token / sent_at

家計と実績

transactions

amount, transaction_type (income/expense) , category, date (datetime)

budgets

year, month, amount。user × 年月で unique

savings_goals

title, target_amount, current_amount, deadline

achievements

ユーザーごとに 20 行を登録時に複製。
original_achievement_id, progress / progress_target, unlocked_at, tier, category

認証とその他

refresh_tokens

token_digest (平文は持たない) , expires_at, revoked

oauth_providers

provider + uid で unique。1 ユーザーに複数可

contacts

name, email, subject, message, status (pending/resolved)。user 紐付けなし

user_actions

定義だけあって誰も書かない・読まない (#406)

掲示板

posts

title, content, likes_count / comments_count / views_count (カウんタ列)

comments

post_id, content, likes_count

likes

polymorphic (likeable_type: Post / Comment)。user × 対象で unique

bookmarks · categories · post_categories

bookmark は user × post で unique。カテゴリは名前で find_or_create し多対多

実績 20 件はどこで進むか

貯金	初めての貯金	収入を 1 件登録	ブロンズ	transactions#create
貯金	マイルストーン ×7	累計収入 1,000 ～ 300,000 円	ブロンズ～プラチナ	create / destroy で再計算
連続記録	3・7・30 日	収入を連日記録 (User#current_streak)	ブロンズ～ゴールド	取引の create ごと
支出管理	予算 ×3	予算を設定 / 月内に収める / 3 ヶ月連続	ブロンズ～ゴールド	budgets#create, 支出の create
特別	目標達成者	貯金目標の current $\geq$ target	ゴールド	savings_goals#update
特別	安心の備え	10 万円以上の貯金目標を作成	プラチナ	savings_goals#create
特別	投資家デビュー	category = investment の収入	ゴールド	transactions#create
特別	初めての投稿	掲示板に 1 件投稿	ブロンズ	posts#create
特別	いいね 10・50	自分の投稿への累計いいね	ブロンズ / シルバー	likes#create_post_like (投稿者側)

「貯金」 = income タイプの取引。給料を登録すると貯金扱いになる (README の定義どおり)

予算 ×3 と目標 ×2 は画面が無いので今は解除不能 (#398)。連続記録は閾値まで進捗 0 のまま (#402)

解除から画面まで: 1 回の取引登録で起きること

1. POST /transactions

保存 → AchievementService で貯金・連続記録・予算の実績を更新



2. 差分を応答に同梱

更新前後の unlocked を比べ、newly_unlocked_achievements として返す



3. フロントで祝う

トースト、解除モーダル（キューで順番に 6 秒）、カワウソが excited に 6 秒



4. 成長ステージを取り直す

useAchievements.refetch({silent}) → summary.growth_stage → OtterAnimation
のリングとバッジが変わる

none	0 ~ 2
bronze	3 ~ 7
silver	8 ~ 12
gold	13 ~ 17
platinum	18 以上（今は届かない）

excited	解除直後 6 秒
sleeping	前回サインインから 7 日以上
happy	今月の貯蓄率が 20% 超
sad	今月が赤字
neutral	それ以外（収入 0 も含む）

数字で見る現在地

5,940

行・front/src (ui とテスト除く)

2,204

行・back/app

46

API アクション・15 コントローラー

37 + 15

RSpec ファイル + Jest ファイル

804

コミット・2024-12 開始

18

open issue・うち今回の監査 16 件